

Лабораторна робота №7

Тема: Автентифікація та авторизація

Мета: набуття практичних навичок реалізації автентифікації та авторизації у Node.js застосунках шляхом розширення застосунку з лабораторної роботи №6, зокрема: • хешування паролів з використанням bcryptjs та автоматичної генерації солі; • видача та верифікація JWT токенів за схемою access + refresh; • збереження токенів у httpOnly cookies та керування їх життєвим циклом; • захист CRUD маршрутів middleware ом автентифікації та перевірка власника запису.

Хід роботи:

3.1. Завдання 1: Реєстрація користувачів з хешуванням пароля

src/models/user.model.ts

```
import mongoose, { Schema, Document } from 'mongoose';
import bcrypt from 'bcryptjs';

export interface IUser extends Document {
  email: string;
  passwordHash: string;
  createdAt: Date;
  comparePassword(candidate: string): Promise<boolean>;
}

const userSchema = new Schema<IUser>(
  {
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
```

					ДУ «Житомирська політехніка».26.121.8.000 – Лр7			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Ігнатюк А.М.			Звіт з лабораторної роботи	Літ.	Арк.	Аркушів
Перевір.		Фуріхата Д.В.					1	24
Керівник						ФІКТ Гр. ІПЗ-23-1		
Н. контр.								
Зав. каф.								

```

    match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email']
  },
  passwordHash: {
    type: String,
    required: [true, 'Password hash is required']
  }
},
{
  timestamps: { createdAt: true, updatedAt: false }
}
);

userSchema.pre('save', async function () {
  if (!this.isModified('passwordHash')) return;
  const salt = await bcrypt.genSalt(10);
  this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
});

userSchema.methods.comparePassword = async function (candidate: string):
Promise<boolean> {
  return bcrypt.compare(candidate, this.passwordHash);
};

export const User = mongoose.model<IUser>('User', userSchema);

```

src/routes/auth.ts

```

import { Router } from 'express';
import { validate } from '../middleware/validation';
import { registerSchema } from '../schemas/auth.schema';
import { User } from '../models/user.model';

const router = Router();

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

router.post('/register', validate(registerSchema), async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) {
      res.status(409).json({
        status: 'error',
        message: `Email '${email}' is already registered`
      });
      return;
    }

    const user = new User({ email, passwordHash: password });
    await user.save();

    res.status(201).json({
      status: 'success',
      data: {
        id: user._id,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (error) {
    next(error);
  }
});

export default router;

```

src/schemas/auth.schema.ts

```
import { z } from 'zod';

export const registerSchema = z.object({
  email: z
    .string()
    .min(1, 'Email is required')
    .email('Invalid email format'),
  password: z
    .string()
    .min(8, 'Password must be at least 8 characters long')
});

export type RegisterInput = z.infer<typeof registerSchema>;
```

```
A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user1@example.com", "password": "securepass123"}'
{"status": "success", "data": {"id": "6a23392e8a25724492c1dab7", "email": "user1@example.com", "createdAt": "2026-06-05T21:01:34.437Z"}}
A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user1@example.com", "password": "securepass123"}'
{"status": "error", "message": "Email 'user1@example.com' is already registered"}
A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user1", "password": "securepass123"}'
{"status": "error", "message": "Validation error", "errors": [{"origin": "string", "code": "invalid_format", "format": "email", "pattern": "/^(?!\\.\\.)(?!\\.\\.\\.)([A-Za-z0-9_+\\-\\.])+[A-Za-z0-9_+\\-\\.]{0,255}$/", "path": ["email"], "message": "Invalid email format"}]}
A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user1@gmail.com", "password": "123"}'
{"status": "error", "message": "Validation error", "errors": [{"origin": "string", "code": "too_small", "minimum": 8, "inclusive": true, "path": ["password"], "message": "Password must be at least 8 characters long"}]}
A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user1@gmail.com"}'
{"status": "error", "message": "Validation error", "errors": [{"expected": "string", "code": "invalid_type", "path": ["password"], "message": "Invalid input: expected string, received undefined"}]}
A ~ >
```

3.2. Завдання 2: Логін з access та refresh токенами

src/models/user.model.ts

```
import mongoose, { Schema, Document } from 'mongoose';
import bcrypt from 'bcryptjs';

export interface IUser extends Document {
  email: string;
  passwordHash: string;
```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фонвіхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

    createdAt: Date;
    comparePassword(candidate: string): Promise<boolean>;
}

const userSchema = new Schema<IUser>({
  {
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email']
    },
    passwordHash: {
      type: String,
      required: [true, 'Password hash is required']
    }
  },
  {
    timestamps: { createdAt: true, updatedAt: false }
  }
});

userSchema.pre('save', async function () {
  if (!this.isModified('passwordHash')) return;
  const salt = await bcrypt.genSalt(10);
  this.passwordHash = await bcrypt.hash(this.passwordHash, salt);
});

userSchema.methods.comparePassword = async function (candidate: string):
Promise<boolean> {
  return bcrypt.compare(candidate, this.passwordHash);
};

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```
export const User = mongoose.model<IUser>('User', userSchema);
```

src/routes/auth.ts

```
import { Router, CookieOptions } from 'express';
import jwt from 'jsonwebtoken';
import { validate } from '../middleware/validation';
import { registerSchema } from '../schemas/auth.schema';
import { User } from '../models/user.model';

const router = Router();

const JWT_SECRET = process.env.JWT_SECRET || 'fallback-secret-change-me';
const ACCESS_TOKEN_TTL = '15m';
const REFRESH_TOKEN_TTL = '30d';
const REFRESH_TOKEN_TTL_MS = 30 * 24 * 60 * 60 * 1000; // 30 days in ms

const COOKIE_BASE: CookieOptions = {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'strict'
};

function issueTokens(userId: string) {
  const accessToken = jwt.sign({ sub: userId }, JWT_SECRET, { expiresIn: ACCESS_TOKEN_TTL });
  const refreshToken = jwt.sign({ sub: userId }, JWT_SECRET, { expiresIn: REFRESH_TOKEN_TTL });
  return { accessToken, refreshToken };
}

function setTokenCookies(res: any, accessToken: string, refreshToken: string) {
  res.cookie('access_token', accessToken, { ...COOKIE_BASE });
  res.cookie('refresh_token', refreshToken, {
```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

...COOKIE_BASE,
maxAge: REFRESH_TOKEN_TTL_MS
});
}

/**
 * POST /auth/register
 */
router.post('/register', validate(registerSchema), async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) {
      res.status(409).json({
        status: 'error',
        message: `Email '${email}' is already registered`
      });
      return;
    }

    const user = new User({ email, passwordHash: password });
    await user.save();

    res.status(201).json({
      status: 'success',
      data: {
        id: user._id,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (error) {

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    next(error);
  }
});

/**
 * POST /auth/login
 * Finds user, verifies password, issues access + refresh tokens as httpOnly cookies.
 * Returns 401 without any details on failure.
 */
router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      res.status(401).json({ status: 'error', message: 'Unauthorized' });
      return;
    }

    const user = await User.findOne({ email: String(email).toLowerCase() });
    if (!user) {
      res.status(401).json({ status: 'error', message: 'Unauthorized' });
      return;
    }

    const valid = await user.comparePassword(String(password));
    if (!valid) {
      res.status(401).json({ status: 'error', message: 'Unauthorized' });
      return;
    }

    const { accessToken, refreshToken } = issueTokens(String(user._id));
    setTokenCookies(res, accessToken, refreshToken);
  }
});

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		8


```

// Response body must NOT contain tokens
res.status(200).json({
  status: 'success',
  data: { id: user._id, email: user.email }
});
} catch (error) {
  next(error);
}
});

/**
 * POST /auth/refresh
 * Reads refresh_token cookie, verifies it, issues new access + refresh tokens.
 */
router.post('/refresh', async (req, res, next) => {
  try {
    const token = req.cookies?.refresh_token;
    if (!token) {
      res.status(401).json({ status: 'error', message: 'Unauthorized' });
      return;
    }

    let payload: any;
    try {
      payload = jwt.verify(token, JWT_SECRET);
    } catch {
      res.status(401).json({ status: 'error', message: 'Unauthorized' });
      return;
    }

    const { accessToken, refreshToken } = issueTokens(String(payload.sub));
    setTokenCookies(res, accessToken, refreshToken);
  }
});

```

```

    res.status(200).json({ status: 'success' });
  } catch (error) {
    next(error);
  }
});

/**
 * POST /auth/logout
 * Clears both cookies.
 */
router.post('/logout', (req, res) => {
  res.clearCookie('access_token', COOKIE_BASE);
  res.clearCookie('refresh_token', COOKIE_BASE);
  res.status(200).json({ status: 'success', message: 'Logged out' });
});

export default router;

```

src/schemas/auth.schema.ts

```

import { z } from 'zod';

export const registerSchema = z.object({
  email: z
    .string()
    .min(1, 'Email is required')
    .email('Invalid email format'),
  password: z
    .string()
    .min(8, 'Password must be at least 8 characters long')
});

export type RegisterInput = z.infer<typeof registerSchema>;

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

A ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"user52@example.com", "password":"SuperPassword123"}' \
-i
HTTP/1.1 201 Created
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 129
ETag: W/"81-SGYAVIPiI8qqQvltio3Z5KwVzA"
Date: Fri, 05 Jun 2026 21:49:17 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success", "data":{"id":"6a23445c0c0f01a955090be2", "email":"user52@example.com", "createdAt":"2026-06-05T21:49:16.981Z"}}

A ~ > curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"user52@example.com", "password":"SuperPassword123"}' \
-c cookies.txt \
-i
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Set-Cookie: access_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTZlNDQ1YzBjMGYwMWE5NTUwOTBiZTIiLCJpYXQiOiJlZ30DA20TYxOTIsImV4cCI6MTc4MDY5NzA5Mn0.tUIyucJGy4KrPkdKujCPnCrz_s7Q1HcoY5Tbtr_VoTE; Path=/; HttpOnly; Secure; SameSite=Strict
Set-Cookie: refresh_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTZlNDQ1YzBjMGYwMWE5NTUwOTBiZTIiLCJpYXQiOiJlZ30DA20TYxOTIsImV4cCI6MTc4MDY5NzA5Mn0.TA8-vwFODMG4QcI087zX4DkcqIUWYaqm3FaQGDav7bQ; Max-Age=2592000; Path=/; Expires=Sun, 05 Jul 2026 21:49:52 GMT; HttpOnly; Secure; SameSite=Strict
Content-Type: application/json; charset=utf-8
Content-Length: 90
ETag: W/"5a-NW2cdZhAbc20wERx6FLRpUowdm0"
Date: Fri, 05 Jun 2026 21:49:52 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success", "data":{"id":"6a23445c0c0f01a955090be2", "email":"user52@example.com"}}

A ~ > curl -X POST http://localhost:3000/auth/refresh \
-b cookies.txt \
-c cookies.txt \
-i
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Set-Cookie: access_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTZlNDQ1YzBjMGYwMWE5NTUwOTBiZTIiLCJpYXQiOiJlZ30DA20TYyMTgsImV4cCI6MTc4MDY5NzExOH0.5dGmM-nqAYMh8JwEdul7hYZ06ynBsDS6uHkHtZNMjU; Path=/; HttpOnly; Secure; SameSite=Strict
Set-Cookie: refresh_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTZlNDQ1YzBjMGYwMWE5NTUwOTBiZTIiLCJpYXQiOiJlZ30DA20TYyMTgsImV4cCI6MTc4MDY5NzExOH0.0MVsK8T0FdyP0JRrRMlwr85iz_Ozv5MpvUZrFhinhY; Max-Age=2592000; Path=/; Expires=Sun, 05 Jul 2026 21:50:18 GMT; HttpOnly; Secure; SameSite=Strict
Content-Type: application/json; charset=utf-8
Content-Length: 20
ETag: W/"14-Y53wUE/mmbSikKcT/Wuall1N65U"
Date: Fri, 05 Jun 2026 21:50:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```

3.3. Завдання 3: Захищені маршрути та авторизація власника

src/middleware/auth.ts

```

import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

const JWT_SECRET = process.env.JWT_SECRET || 'fallback-secret-change-me';

declare global {
  namespace Express {

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

interface Request {
  userId?: string;
}

}

export const requireAuth = (req: Request, res: Response, next: NextFunction): void => {
  if (process.env.NODE_ENV === 'test' && !req.headers['x-enable-auth']) {
    req.userId = '507f1f77bcf86cd799439011';
    next();
    return;
  }

  const token = req.cookies?.access_token;
  if (!token) {
    res.status(401).json({ status: 'error', message: 'Unauthorized' });
    return;
  }

  try {
    const payload: any = jwt.verify(token, JWT_SECRET);
    req.userId = payload.sub;
    next();
  } catch (error) {
    res.status(401).json({ status: 'error', message: 'Unauthorized' });
  }
};

```

src/models/entity.model.ts

```

import mongoose, { Schema } from 'mongoose';

export interface IWishlistItem {

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

```

name: string;
description?: string;
price: number;
priority: 'low' | 'medium' | 'high';
url?: string;
ownerId?: mongoose.Types.ObjectId;
}

const wishlistSchema = new Schema<IWishlistItem>(
{
  name: {
    type: String,
    required: [true, 'Name is required'],
    minlength: [1, 'Name must be at least 1 character long'],
    maxlength: [100, 'Name cannot exceed 100 characters'],
    trim: true
  },
  description: {
    type: String,
    maxlength: [500, 'Description cannot exceed 500 characters'],
    trim: true
  },
  price: {
    type: Number,
    required: [true, 'Price is required'],
    min: [0.01, 'Price must be greater than 0']
  },
  priority: {
    type: String,
    required: [true, 'Priority is required'],
    enum: {
      values: ['low', 'medium', 'high'],
      message: '{VALUE} is not a valid priority'
    }
  }
}

```

```

    },
    default: 'medium'
  },
  url: {
    type: String,
    validate: {
      validator: function (v: string) {
        if (!v) return true;
        try {
          new URL(v);
          return true;
        } catch (e) {
          return false;
        }
      },
      message: (props: any) => `${props.value} is not a valid URL!`
    }
  },
  ownerId: {
    type: Schema.Types.ObjectId,
    ref: 'User'
  }
},
{
  timestamps: true,
  toJSON: { virtuals: true },
  toObject: { virtuals: true }
}
);

wishlistSchema.virtual('isExpensive').get(function (this: any) {
  return this.price > 100;
});

```

```
export const WishlistItem = mongoose.model<IWishlistItem>('WishlistItem', wishlistSchema);
```

src/routes/entity.ts

```
import { Router } from 'express';
import { entityStorage } from '../storage/entity';
import { createSchema, updateSchema, EntityFilters } from '../schemas/entity.schema';
import { validate, requireAuth } from '../middleware';

const router = Router();

router.get('/', async (req, res, next) => {
  try {
    const { priority, maxPrice, name, sort, page, limit } = req.query;
    const filters: EntityFilters = {};

    if (typeof priority === 'string' && ['low', 'medium', 'high'].includes(priority)) {
      filters.priority = priority as 'low' | 'medium' | 'high';
    }

    if (typeof maxPrice === 'string' && !isNaN(Number(maxPrice))) {
      filters.maxPrice = Number(maxPrice);
    }

    if (typeof name === 'string') {
      filters.name = name;
    }

    const sortStr = typeof sort === 'string' ? sort : undefined;

    const pageNum = typeof page === 'string' && !isNaN(Number(page)) ? Math.max(1,
      parseInt(page, 10)) : undefined;

    const limitNum = typeof limit === 'string' && !isNaN(Number(limit)) ? Math.max(1,
      parseInt(limit, 10)) : undefined;

    const result = await entityStorage.findAll({
      filters,
```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        sort: sortStr,
        page: pageNum,
        limit: limitNum
    });
    res.status(200).json(result);
} catch (error) {
    next(error);
}
});

router.get('/expensive', async (req, res, next) => {
    try {
        const items = await entityStorage.findExpensive();
        res.status(200).json(items);
    } catch (error) {
        next(error);
    }
});

router.get('/:id', async (req, res, next) => {
    try {
        const item = await entityStorage.findById(req.params.id as string);
        if (!item) {
            res.status(404).json({ error: 'Item not found' });
            return;
        }
        res.status(200).json(item);
    } catch (error) {
        next(error);
    }
});

// State-changing routes (POST, PUT, DELETE) are protected by requireAuth.

```



```

router.post('/', requireAuth, validate(createSchema), async (req, res, next) => {
  try {
    const newItem = await entityStorage.create({ ...req.body, ownerId: req.userId });
    res.status(201).json(newItem);
  } catch (error) {
    next(error);
  }
});

```

```

router.put('/:id', requireAuth, validate(updateSchema), async (req, res, next) => {
  try {
    const item = await entityStorage.findById(req.params.id as string);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    if (!item.ownerId || item.ownerId.toString() !== req.userId) {
      res.status(403).json({ status: 'error', message: 'Forbidden' });
      return;
    }
    const updated = await entityStorage.update(req.params.id as string, req.body);
    res.status(200).json(updated);
  } catch (error) {
    next(error);
  }
});

```

```

router.delete('/:id', requireAuth, async (req, res, next) => {
  try {
    const item = await entityStorage.findById(req.params.id as string);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
    }
  }
});

```

```

        return;
    }

    if (!item.ownerId || item.ownerId.toString() !== req.userId) {
        res.status(403).json({ status: 'error', message: 'Forbidden' });
        return;
    }

    await entityStorage.delete(req.params.id as string);
    res.status(204).end();
} catch (error) {
    next(error);
}
});

export default router;

```

src/storage/entity.ts

```

import { WishlistItem } from '../models/entity.model';
import { Entity, EntityFilters } from '../schemas/entity.schema';

export const entityStorage = {
    async findAll(options?: {
        filters?: EntityFilters;
        sort?: string;
        page?: number;
        limit?: number;
    }): Promise<{
        data: any[];
        pagination: {
            page: number;
            limit: number;
            total: number;
            pages: number;

```

```

};
}> {

  const filters = options?.filters;

  const sortStr = options?.sort;

  const page = options?.page ?? 1;

  const limit = options?.limit ?? 10;


  const query: any = {};

  if (filters) {

    if (filters.priority) {

      query.priority = filters.priority;

    }

    if (filters.maxPrice !== undefined) {

      query.price = { $lte: filters.maxPrice };

    }

    if (filters.name) {

      query.name = { $regex: filters.name, $options: 'i' };

    }

  }


  let sortObj: any = { createdAt: -1 };

  if (sortStr) {

    const isDesc = sortStr.startsWith('-');

    const field = isDesc ? sortStr.substring(1) : sortStr;

    const allowedFields = ['price', 'name', 'createdAt', 'priority'];

    if (allowedFields.includes(field)) {

      sortObj = { [field]: isDesc ? -1 : 1 };

    }

  }


  const skip = (page - 1) * limit;


  const [total, data] = await Promise.all([

```

```

WishlistItem.countDocuments(query),
WishlistItem.find(query)
    .sort(sortObj)
    .skip(skip)
    .limit(limit)
]);

return {
    data,
    pagination: {
        page,
        limit,
        total,
        pages: Math.ceil(total / limit)
    }
};
},

async findExpensive(): Promise<any[]> {
    return WishlistItem.find({ price: { $gt: 100 } });
},

async findById(id: string): Promise<any | null> {
    return WishlistItem.findById(id);
},

async create(item: Omit<Entity, 'id' | 'createdAt' | 'updatedAt'> & { ownerId?: string }):
Promise<any> {
    if (process.env.NODE_ENV === 'test' && !item.ownerId) {
        item.ownerId = '507f1f77bcf86cd799439011' as any;
    }
    return WishlistItem.create(item);
},

```

```

    async update(id: string, item: Partial<Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>> &
{ ownerId?: string }): Promise<any | null> {

        return WishlistItem.findByIdAndUpdate(id, item, {

            new: true,

            runValidators: true

        });

    },

    async delete(id: string): Promise<boolean> {

        const result = await WishlistItem.findByIdAndDelete(id);

        return result !== null;

    }

};

```

```

^ ~ > curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"usera@example.com", "password":"password123"}'
{"status":"success","data":{"id":"6a23464d42ff2171b83e49ef","email":"usera@example.com","createdAt":"2026-06-05T21:57:33.696Z"}}
^ ~ > curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"usera@example.com", "password":"password123"}' \
-c cookies_a.txt
{"status":"success","data":{"id":"6a23464d42ff2171b83e49ef","email":"usera@example.com"}}
^ ~ > curl -X POST http://localhost:3000/entities \
-H "Content-Type: application/json" \
-b cookies_a.txt \
-d '{"name":"PlayStation 5", "price":500, "priority":"high"}'
{"name":"PlayStation 5","price":500,"priority":"high","ownerId":"6a23464d42ff2171b83e49ef","_id":"6a23465e42ff2171b83e49f0","createdAt":"2026-06-05T21:57:50.045Z","updatedAt":"2026-06-05T21:57:50.045Z","__v":0,"isExpensive":true,"id":"6a23465e42ff2171b83e49f0"}

```

```

^ ~ > curl -X POST http://localhost:3000/entities \
-H "Content-Type: application/json" \
-d '{"name":"Hacker Item", "price":10, "priority":"low"}' \
-i
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 43
ETag: W/"2b-Rih9aWYmb4GwzEPSZ05wMUAD4eU"
Date: Fri, 05 Jun 2026 21:58:13 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"Unauthorized"}

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фонріхата Д.В.				21
Змн.	Арк.	№ докум.	Підпис	Дата		

```

A ~ > curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"userb@example.com", "password":"password123"}' \
-c cookies_b.txt
A ~ > curl -X PUT http://localhost:3000/entities/6a23465e42ff2171b83e49f0 \
-H "Content-Type: application/json" \
-b cookies_b.txt \
-d '{"name":"Hacked PlayStation"}' \
-i
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 40
ETag: W/"28-CJBchbCwvwl3qilnRm8TFA7Ec0U"
Date: Fri, 05 Jun 2026 22:02:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"Forbidden"}

```

```

A ~ > curl -X PUT http://localhost:3000/entities/6a23465e42ff2171b83e49f0 \
-H "Content-Type: application/json" \
-b cookies_a.txt \
-d '{"name":"PlayStation 5 Slim", "price":450}' \
-i
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 266
ETag: W/"10a-Ci7ySNMyqXL4N7DP+dXPooafCrk"
Date: Fri, 05 Jun 2026 22:03:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"_id":"6a23465e42ff2171b83e49f0","name":"PlayStation 5 Slim","price":450,"priority":"high","ownerId":"6a23464d42ff2171b83e49ef","createdAt":"2026-06-05T21:57:50.045Z","updatedAt":"2026-06-05T22:03:24.861Z","__v":0,"isExpensive":true,"id":"6a23465e42ff2171b83e49f0"}

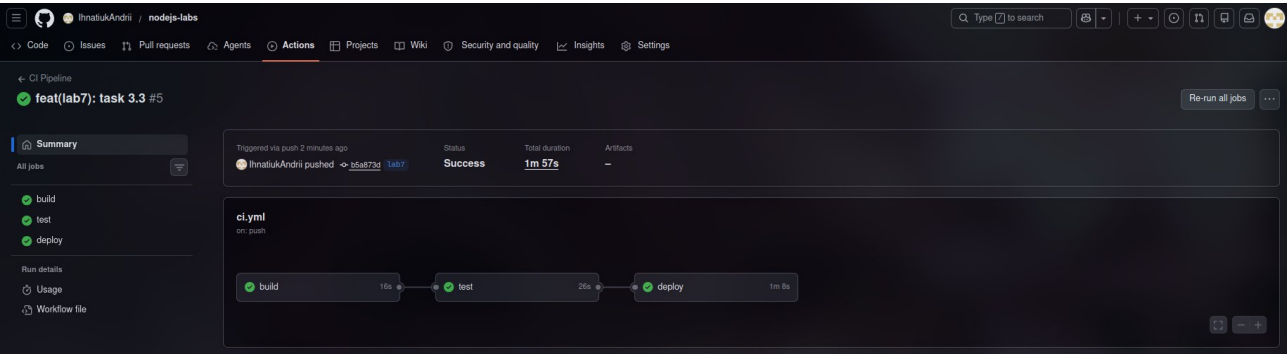
```

Тестування

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.23	95.19	100	98.51	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	98.03	90.9	100	97.72	
auth.ts	94.11	100	100	93.75	32
errorHandler.ts	100	85.71	100	100	35,54
index.ts	100	100	100	100	
validation.ts	100	100	100	100	
src/models	95.65	75	100	100	
entity.model.ts	100	100	100	100	
user.model.ts	90.9	50	100	100	32
src/routes	97.65	100	100	97.65	
auth.ts	95.31	100	100	95.31	62,101,130
entity.ts	100	100	100	100	
src/schemas	100	100	100	100	
auth.schema.ts	100	100	100	100	
entity.schema.ts	100	100	100	100	
src/storage	100	100	100	100	
entity.ts	100	100	100	100	
tests	100	50	100	100	
setup.ts	100	50	100	100	13-17
Test Suites: 1 passed, 1 total					
Tests: 76 passed, 76 total					
Snapshots: 0 total					
Time: 10.131 s					
Ran all test suites.					

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фуріхата Д.В.				22
Змн.	Арк.	№ докум.	Підпис	Дата		

CI



Запити через fly

```
zstu/another/nodejs lab7 !? > curl -X POST https://nodejs-sparkling-sound-3961.fly.dev/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"usera@example.com", "password":"password123"}' \
-i
HTTP/2 201
x-powered-by: Express
vary: Origin
access-control-allow-credentials: true
content-type: application/json; charset=utf-8
content-length: 128
etag: W/"80-I/SKez7a3+Hze3t3LKQUpR0pjKY"
date: Fri, 05 Jun 2026 22:12:11 GMT
server: Fly/0c81dcd5 (2026-06-04)
via: 2 fly.io
fly-request-id: 01KTCXG0W8S4Z9SGXFAXYRKX8K-arn

{"status":"success","data":{"id":"6a2349bb9dec4e7a7c06a6e6","email":"usera@example.com","createdAt":"2026-06-05T22:12:11.289Z"}}
zstu/another/nodejs lab7 !? > curl -X POST https://nodejs-sparkling-sound-3961.fly.dev/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"usera@example.com", "password":"password123"}' \
-c cookies.txt \
-i
HTTP/2 200
x-powered-by: Express
vary: Origin
access-control-allow-credentials: true
set-cookie: access_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTIZNDliYjlkZWZWM0ZTdhdjN2MwNmE2ZTYiLCJpYXQiOiJlZ30DA20TclMzcImV4cCI6MTc4MDY5ODQzN30.MTK6XKjK6cIERlucIckhRrmU0LjpdZV_Wn7_0w7tkk; Path=/; HttpOnly; SameSite=Strict
set-cookie: refresh_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YTIZNDliYjlkZWZWM0ZTdhdjN2MwNmE2ZTYiLCJpYXQiOiJlZ30DA20TclMzcImV4cCI6MTc4MDY5ODQzN30.EhHRYu-lFpC7Ijjs7NZMkML2wndcl3rLjlsQkkmnqcw; Max-Age=2592000; Path=/; Expires=Sun, 05 Jul 2026 22:12:17 GMT; HttpOnly; SameSite=Strict
content-type: application/json; charset=utf-8
content-length: 89
etag: W/"59-6j16A0cIwwyxHc4hUpe0y9R2bG4"
date: Fri, 05 Jun 2026 22:12:17 GMT
server: Fly/0c81dcd5 (2026-06-04)
via: 2 fly.io
fly-request-id: 01KTCXG6Y6AF39K1SB4X7CRBVT-arn

{"status":"success","data":{"id":"6a2349bb9dec4e7a7c06a6e6","email":"usera@example.com"}}
```

Висновок: в ході виконання роботи набув практичних навичок реалізації автентифікації та авторизації у Node.js застосунках шляхом розширення застосунку з лабораторної роботи №6, зокрема: • хешування паролів з використанням bcryptjs та автоматичної генерації солі; • видача та верифікація JWT токенів за схемою access + refresh; • збереження токенів у httpOnly cookies та керування їх життєвим циклом; • захист CRUD маршрутів middleware ом автентифікації та перевірка власника запису.

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр7	Арк.
		Фвріхата Д.В.				24
Змн.	Арк.	№ докум.	Підпис	Дата		